



A Model-Based Test Script Generation Framework and Industrial Insight

Muhammad Nouman Zafar¹ · Wasif Afzal¹ · Eduard Paul Enoiu¹ · Zulqarnain Haider² · Inderjeet Singh²

Received: 31 October 2023 / Accepted: 21 February 2025
© The Author(s) 2025

Abstract

Model-based testing (MBT) generates test cases through a model representing the software under test (SUT). The generated abstract test cases need to be transformed into concrete or executable test scripts. Despite the benefits offered by MBT, its industrial adoption is slow. This paper aims to propose a Model-Based Test script GenERation fRamework (TIGER) based on GraphWalker (GW), an open-source MBT tool, to evaluate the accuracy of generated test scripts to reflect real-world scenarios defined by the model, and to report on the findings of an industrial survey on MBT adoption. We have validated the robustness of the TIGER using an industrial case study from Alstom Rail AB, Sweden. We have injected faults into the model of the SUT based on three mutation operators to generate faulty test scripts. The aim of generating faulty test scripts is to produce failing test steps and to guarantee the absence of faults in the SUT. Moreover, we have also generated the test scripts using the correct version of the model and executed them to analyze the behavior of the generated test scripts in comparison with manually written test scripts. The experimental results show that the generated test scripts are executable, provide 100% requirements coverage, and can be used to uncover faults at the software-in-the-loop simulation level of system testing. Additionally, the analysis of the survey data reveals that MBT can address most of the identified testing challenges, but there remain certain barriers to its adoption.

Keywords Model-based testing · Test script generation · Case study · Industrial survey

Introduction

Safety-critical embedded system software is an essential part of modern cyberphysical systems (CPSs), and errors in CPS software can lead to catastrophic events. Software testing for safety-critical embedded system software needs to be thorough and effective. Research into Model-Based

Testing (MBT) [1] has shown that it can be a resource-efficient technique for generating test cases, test scripts, and test verdicts, as well as effective in terms of fault detection. MBT generates abstract test cases based on a model of the System Under Test (SUT) [2]. The abstract test cases can be transformed into concrete, executable test scripts that can eventually produce test verdicts. However, we have observed that this concretization step is not covered well enough in the literature or is discussed only in the context of web-based and mobile applications where somewhat mature script generation frameworks exist, such as Selenium and Appium. Thus, only a few papers (i.e. [3, 4]) report on the details of the concretization step and even fewer have addressed it for embedded software testing at integration and system levels.

Despite the benefits that MBT offers in terms of test generation efficiency and fault effectiveness, the industrial adoption of MBT is relatively slow [5]. Different factors such as lack of domain-specific and platform-dependent solutions, switching to a modeling paradigm, and lack of industrial success stories, etc., have been reported as key contributors to its gradual adoption in various studies (e.g., [5, 6]).

✉ Muhammad Nouman Zafar
muhammad.nouman.zafar@mdu.se

Wasif Afzal
wasif.afzal@mdu.se

Eduard Paul Enoiu
eduard.paul.enoiu@mdu.se

Zulqarnain Haider
zulqarnain.haider@alstomgroup.com

Inderjeet Singh
singh.inderjeet@alstomgroup.com

¹ Mälardalen University, Västerås, Sweden

² Alstom Rail Sweden AB, Västerås, Sweden

However, to make a conscious decision about MBT adoption for particular contexts, it is crucial to gather and analyze empirical data, insights, and feedback from industrial practitioners. Hence, we also find it interesting to investigate the challenges and perceptions of MBT among industrial practitioners to assess the level of its adoption and to understand the factors contributing to or limiting the successful industrial adoption of MBT.

In this paper, we have developed a Model-Based Test script GenERation fRamework (TIGER)¹ based on Graph-Walker (GW),² a freely-available MBT tool. The generated test scripts validate the functionality of sub-systems controlled by the train control management system (TCMS) software developed by Alstom Rail, Sweden, AB. GW is utilized for generating abstract test cases that traverse the nodes and edges of the Extended Finite State Machine (EFSM) model representing the SUT. The developed framework uses GW to generate abstract test cases, concretizes them by following the defined mapping rules, and generates test scripts in C# language. To ensure the credibility of TIGER-generated test scripts capturing the intended functionalities defined in the model and identifying their potential weaknesses, we have analyzed the behavior of the generated test scripts by injecting faults into the model of a fire detection system controlled by the TCMS. The injected faults are based on the work on mutation operators by Belli et al. [7]. The test scripts are then generated from the correct and fault-induced version of the models and executed at the software-in-the-loop (SiL) execution platform at Alstom. The aim of introducing faults in the model is to generate faulty test scripts. These should fail when executed on the SUT to guarantee the absence of induced faults in the SUT with no known fault. We have also analyzed the input combinations generated by GW to evaluate the requirements coverage of test scripts. The results show that TIGER can generate test scripts that are readily executable on Alstom's SiL platform and have the potential to uncover interaction faults at the sub-system level of TCMS testing.

The work presented in this paper is an extension of a prior contribution made at a conference [8]. In addition to the previous contribution, we have conducted an industrial survey by collecting qualitative and quantitative data in order to gain an understanding of the industrial viewpoint on the adoption of MBT and associated challenges. We have also discussed the indirect correlation between the survey findings and the practical relevance of TIGER in an industrial setting. Furthermore, we have also extended the related work for an in-depth analysis of the factors contributing to slow

MBT adoption reported in the previously conducted industrial surveys and reports. The results of the industrial survey show the positive expectations of industrial practitioners from MBT in terms of solving the problem of unambiguous requirements through modeling, increased test coverage, and resource reduction through automatic test generation. It also uncovers the challenges influencing the industrial adoption of MBT.

The rest of the article is organized as follows. “**Preliminaries**” presents a brief description of related concepts and studies. “**Proposed Framework**” provides a detailed description of the proposed framework. “**Study Design**” describes the study design, including research questions, experimental design, and survey design. “**Results**” presents the quantitative and qualitative results of the experimentation and industrial survey based on formulated research questions. “**Discussion**” and “**Validity Threats**” discuss the results and threats to the validity of the study, respectively, whereas the conclusions and future work are presented in “**Conclusion and Future Work**”.

Preliminaries

This section briefly introduces model-based testing, Graph-Walker, model-based mutation testing, and qualitative data analysis, along with related work concerning model-based test generation techniques and industrial studies on MBT adoption.

Model-Based Testing

Model-based testing [9] comprises mainly three distinct steps; (i) the creation of a model in conformance with the SUT and requirement specification, (ii) the generation of abstract and concrete test cases, and (iii) the execution of a test suite on the SUT to validate its behavior. Depending on test objectives and test artifacts, various modeling notations and languages (e.g., class diagrams, state machines, and finite automata) can be used to create MBT models representing the abstract view of the system's functionalities and interaction of a SUT. These models serve as a foundation to facilitate abstract test case generation based on a specific generation algorithm and coverage criteria. Test generation algorithms are strategy-based methods such as random, combinatorial, or symbolic execution, which automatically create the test cases systematically. On the other hand, coverage criteria determine the extent to which different functionalities of a SUT need to be exercised (e.g., statement coverage, path coverage, etc.).

The abstract test cases generated from a model do not contain concrete execution information about the SUT, and hence, require an approach (i.e., transformation, adaptation,

¹ TIGER is available at <https://github.com/MuhammadNoumanZafar/TestScriptGeneration.git>.

² <https://github.com/GraphWalker/graphwalker-project/wiki>.

or mixed approach) to bridge the gap between the abstract and real-world values of the model and SUT [9]. In the transformation approach, abstract concepts of the models captured by the abstract test cases are directly converted into actionable and precise values for input data and expected results enriched with implementation-specific details of a SUT [2]. In contrast, the adaptation approach introduces a manual written intermediary adapter code that contains a translation between the abstract test cases generated from the model and corresponding concrete actions executable on a SUT [9]. On the other hand, a mixed approach is a combination of transformation and adaptation approaches. It generates test inputs and expected results with precise implementation-specific values and utilizes a layer of adapter code to execute the tests on a SUT to produce test verdicts.

GraphWalker

GraphWalker (GW) is an open-source model-based testing tool that offers different generation algorithms (e.g., random, weight_random, quick_random, etc.) and stopping conditions (e.g., edge_coverage, vertex_coverage, time_duration, etc.) along with online and offline test generation features. GW comes in three versions: GraphWalker studio, GraphWalker Command Line Interface (CLI), and GW4E (an Eclipse plugin) [10]. Our framework utilizes GW studio to create and validate models and GW CLI to generate abstract test cases.

GW studio provides a user-friendly web-based graphical modeling environment to create an explicit model of a SUT using Extended Finite State Machines (EFSM). An EFSM model in GW comprises extended and shared states, data variables, guard conditions, actions, and transitions. The extended states in an EFSM model incorporate dynamic data and variables. In contrast, shared states facilitate the reusability of states and traversing across different state machines to generate test scenarios. On the other hand, the data variables in an EFSM model hold values between states, guard conditions are boolean expressions depicting the logical conditions based on requirements of a SUT to influence the model's behavior, and transitions represent the dynamic behavior and relation between the states dictated by guard conditions. GW studio also contains model simulation for the validation and analysis of the graphical model based on the provided generation algorithm and stopping condition.

GW CLI³ provides a command-line interface to generate, execute, and manage test cases supporting various formats used to define a model, i.e., GraphML and JSON. It allows a user to specify test generation parameters (i.e., generation

algorithm and stopping condition) to generate test paths, execute test cases, enable online or offline features, and produce test coverage results and reports. It also provides different features to control the levels of details to be generated in test cases, e.g., the generation of test data along with test paths and unvisited elements in models. Furthermore, it can also be incorporated into industrial testing workflows, providing seamless integration with other development tools (e.g., Selenium) [11].

Model-Based Mutation Testing

Model-based mutation testing (MBMT) is a fault-based black box testing technique that involves generating or improving a test suite by making different versions of an original model when the source code of a SUT is inaccessible. These versions can also be referred to as faulty models or mutants. Each mutant contains a small systematic modification representing an error in the SUT based on a specific mutation operator [9]. After creating mutants, MBMT automatically generates test cases containing the scenarios to distinguish the behavior of the original model and mutants [12]. We have used an instance of the MBMT approach proposed by Belli et al. [7] to assess the fault detection capabilities of the test suite generated by TIGER. In traditional MBMT, the generation of test cases aims to kill the non-equivalent mutants through an equivalence check between the original and mutated models [13], whereas Belli's approach focuses on assessing the fault detection capabilities of a test suite generated from an original model in comparison to test suites generated from mutated ones.

In this approach, the behavioral model (M) and its corresponding SUT are assumed to have already been validated by a test suite (T) generated from M through the MBT approach using a specific generation algorithm and coverage criterion. It involves creating mutants (M_{fi}) based on the proposed elementary mutation operators, i.e., insertion and omission, generating test suites (T_{fi}) based on M_{fi} using the similar test generation algorithm and coverage criterion as T , and comparatively analyzing the test results to evaluate the fault effectiveness of generated test suites. With respect to this approach, a mutant (M_{f1}) is said to be killed if the corresponding test suite (T_{f1}) produces failed test steps when executed on the SUT, otherwise, M_{f1} is considered alive. Furthermore, alive mutants could indicate the presence of faults in the SUT that have not been detected by the test suite T .

Qualitative Data Analysis

Qualitative data analysis is a process that systematically examines and interprets non-numeric data to draw implications by understanding the underlying meaning, pattern,

³ <https://github.com/GraphWalker/graphwalker-project/wiki/Command-Line-Tool>.

or themes. Various approaches include thematic analysis, ground theory, and content analysis [14]. We used thematic analysis to qualitatively analyze survey responses and followed Cruzes et al.'s [15] recommended steps. These steps include extraction and initial reading of the data, identifying and coding interesting concepts, translating the codes into sub-themes, exploring the relationship between the sub-themes to create higher-order themes, and assessing the trustworthiness of synthesis.

The first step deals with a thorough reading of the collected data to understand the initial ideas and patterns. In the second step, significant features are identified, and initial codes are assigned to each feature, each of which represents specific data segments related to research objectives. The third step involves the classification of the codes into groups with potential sub-themes. These sub-themes are then explored, interpreted, and classified in a wider context to constitute higher-order themes to achieve defined research objectives in step four. Finally, in the last step, a discussion or arguments are established to provide insights into the most probable meaning of each identified perspective.

Related Work

Over the past few decades, researchers have been exploring multiple techniques to automate the software testing process and MBT is one of them. Automation of software testing involves the generation of test cases and test scripts and the execution of test scripts to generate test verdicts. We have found numerous studies (e.g., [3, 4, 16–24]) that have provided multiple solutions for automatically generating test artifacts (i.e. test cases and test scripts) using MBT techniques, particularly, with behavioral model. Moreover, various studies (i.e., [1, 5, 6, 25]) have also explored the capabilities of MBT or reported the experience of industrial practitioners with respect to its applicability. In this section, we briefly summarize these studies and for a complete overview of MBT techniques in general, we refer the interested reader to several papers on the topic [26–33]. We further summarize studies describing the industrial adoption of MBT in “[Industrial Studies on the Adoption of Model-Based Testing](#)”.

Model-Based Test Generation Techniques

A model-based solution has been presented by Mohacsi et al. [16] to address the testing challenges and to automatically generate the test scripts for distributed Multi-Mission User Services (MMUS) systems. They developed the model representing the SUT, linked it with the GUI, and generated the test cases using a tool named ‘TEMPPO Designer’. The generation of executable test cases/scripts has been done with Micro Focus TestPartner. The results showed

an improvement in test quality, removal of redundancies, reduction in test effort and increase in test efficiency. Wang et al. [17] proposed a model-based framework for Cloud API testing. It uses data modeling, individual API modeling, and API scenario modeling using directed graphs to generate test scripts containing test cases as well as test data. Another MBT framework has been presented by Marinescu et al. [18] to generate test cases and test scripts for embedded systems by extending EAST-ADL with timed automata semantics and also provided a chain of tools for automatically generating and executing the test script for code validation.

Xu et al. [3] proposed a novel automated model-based test script generation technique for functional and security testing. They used Petri-nets to model the control and data flow for the system’s functional dependencies. They also provided a mapping between the model elements and implementation constructs for test script generation and found that the proposed technique is significantly effective in terms of fault detection. An automated framework to generate test cases and scripts for Android applications based on system sequence diagrams has been reported by Anbunathan et al. [19]. The experimental evaluation showed a significant improvement in bug detection along with better effectiveness in terms of test coverage when compared with other methods. Similarly, a test script generation approach has been proposed by Bonfanti et al. [22]. They have proposed a tool that automatically generates the test scripts in C++ by translating the constructs of scenarios written in AVALLA language based on some defined rules.

Gupta et al. [20] and Moura et al. [21] developed model-driven approaches for generating test cases and scripts based on Business Process Modelling Notations (BPMN) focused on functional testing for web-based applications. The empirical evaluation not only showed the completeness and correctness of the proposed approaches but also turned out to be efficient in terms of performance. Similarly, a model-based approach for generating test cases and test scripts (according to Robot Framework) from Requirements Specification Language (RSL) has been proposed by Paiva et al. [4]. They have also defined a mapping for RSL constructs and GUI elements for the concretization of test data in test scripts. The results showed that the use of RSL improved the quality of requirements and reduced the manual effort and time required to generate test artifacts.

Vanhecke et al. [23] developed a plugin to concretize abstract test cases by using a transformation and adaption approach. They have also provided the mapping of operations and verification with corresponding assertions and actions defined in abstract test cases to generate executable Python scripts in QTaste format. The results show that the plugin increased the generality of QTaste by raising the abstraction level of the SUT interface and the test API. Similarly, a test reuse strategy to minimize the concretization

effort has been proposed by Fragal et al. [24]. After the concretization of FSM-based generated abstract test cases, they used a selection algorithm to select the non-redundant concrete test cases, which can be used to test the unchanged behavior of a new product. The results showed that the reusability of concretized test cases not only reduced the effort but also minimized the cost of testing.

Industrial Studies on the Adoption of Model-Based Testing

An industrial survey to assess the expectations of industrial practitioners from model-based testing has been presented by Kriebel et al. [25]. The results of the survey show a general positive expectation of industrial practitioners in terms of high-quality test case generation. Based on the survey results, the authors also proposed a model-based methodology and created a toolchain to improve the quality of test cases for industrial systems used by Bayerische Motoren Werke AG (BMW), a luxury vehicle manufacturer. Another interview survey with an aim to provide best practices guidelines for MBT has been reported by Alegroth et al. [6]. The results of the survey suggest 23 technical and non-technical guidelines for utilizing MBT at an organizational level. It also identified the need for more experience reports and success stories for its successful adoption.

Binder et al. [1] conducted a series of follow-up surveys to evaluate the effectiveness and efficiency of MBT in comparison to the expectations of the practitioners with MBT. The results show the positive effects of diverse MBT approaches on the effectiveness and efficiency of testing activities, however, MBT fulfills only a fraction of the extremely high expectations of practitioners. The reported expectations include the efficient and early design of the system, managing the complexity of the system and efficient communication with stakeholders through modeling, and reusability of models for test generation. Similarly, Kholkar et al. [5] provided an experience report on deploying MBT for the user acceptance testing of industrial embedded systems. The results of the study show an increase of 20–30% efficiency in designing and planning test activities as well as generating high-quality and effective tests. It also provides evidence of a high potential to pay back the cost in the long term.

Research Gap

The reviewed model-based test generation approaches were often found inadequate to comprehensively address the challenges of state-dependent behavior, real-time constraints, and complex interactions between different components of *embedded software*, especially at the *system level*. Moreover, we have found only a few studies that provide high-level details of the concretization process to generate executable

test scripts. Hence, we argue that the increasing interaction complexity of embedded software at the system level necessitates proposing, developing, and validating a new testing framework.

Proposed Framework

The proposed Model-Based Test script GenERation fRamework (TIGER) is intended to provide cost and time-efficient solution to automatically test embedded software at the system level. The framework consists of three main parts (Fig. 1):

1. Abstract test case generator
2. Test case concretizer
3. Test script generator.

In the text below, we describe each of these main parts of TIGER in detail with respective constituent steps.

Abstract Test Case Generator

The framework is based on GraphWalker (GW), which is used to generate abstract test cases from a model representing a SUT. These test cases do not explicitly contain the precise information about the input/output signals involved in the operation of the SUT and thus can not be executed directly on the SUT to verify the correctness of results [2]. GW uses a model file in JSON or GraphML format and generates the abstract test cases in the form of multiple test steps by traversing through the model elements (i.e. nodes and edges) using a generator and a stopping condition. It provides various options that could be used to generate different information about the model in test cases. For example, one could generate test cases without including the test data (variables and their respective values). In this case, the test cases generated through GW can only contain the names of the traversed nodes and edges. On the other hand, options to include test data and filtering blocked elements could be used to include several pieces of information related to the model in test cases. However, in our case, the `verbose` feature for generating test cases is useful as it generates all the relevant details related to the model in test cases, such as initialized variables, actions, and properties of nodes and edges. This information, along with test paths, helps in identifying the combination of input values, expected outputs, and timing constraints to validate a system's behaviour.

Format of Abstract Test Cases

GW generates the abstract test cases in a JSON format. Figure 2 shows a test step in the JSON format generated using

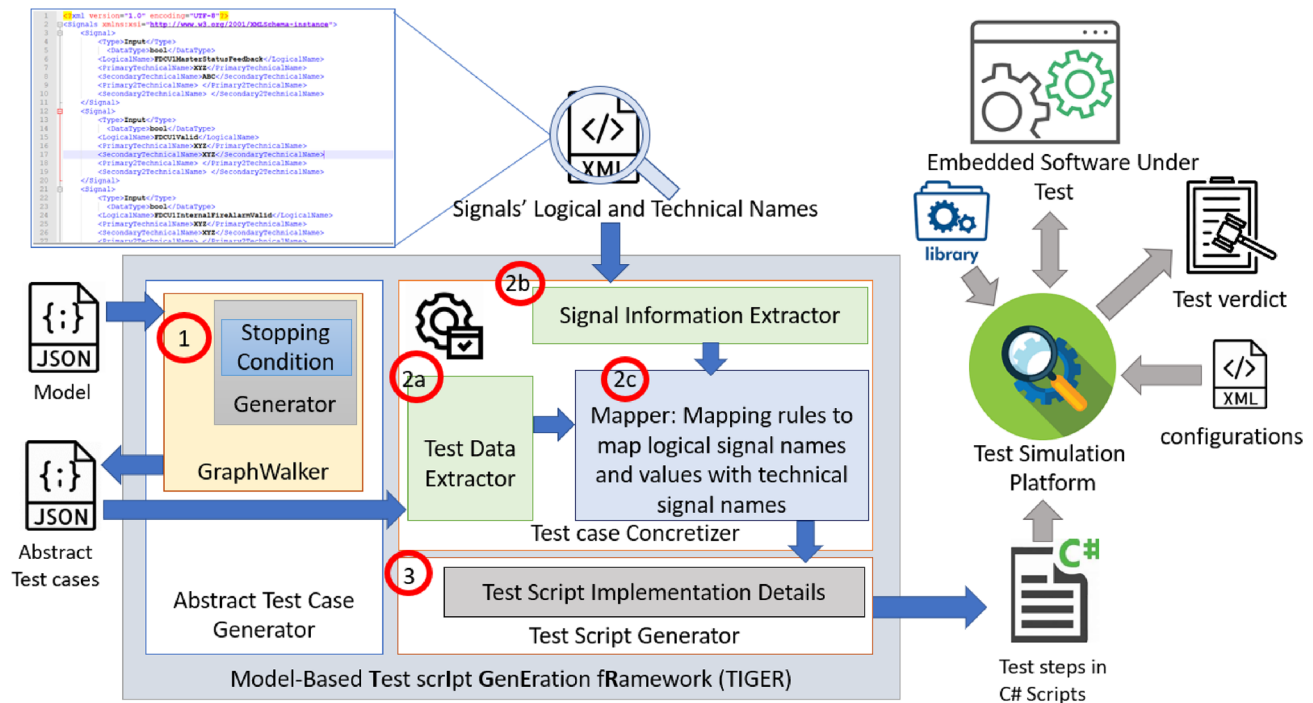


Fig. 1 The architecture of the Model-Based Test script GenERation fRamework (TIGER)

```

{
  "modelName": "FDCU1Model",
  "data": [
    { "Variable1": "true" },
    { "Variable2": "true" },
    { "Variable3": "true" },
    { "Variable4": "false" },
    { "Variable5": "false" },
    { "Variable6": "false" },
    { "Variable7": "true" },
    { "Variable8": "true" },
    { "Variable9": "false" },
    { "Variable10": "true" },
    { "Variable11": "true" },
    { "Variable12": "true" }
  ],
  "currentElementID": "0759c800-fc02-11ea-bbed-d706226969ed",
  "currentElementName": "FDCU1",
  "actions": [ { "Action": "global.Variable1 = false;" } ],
  "properties": [
    { "x": 318 },
    { "y": 322 }
  ]
}

```

Fig. 2 Example of a JSON formatted test step generated by GW

the verbose option. The model name in the JSON object (“modelName”) contains the name of the model that GW traversed while generating this test step, and “data” holds the names of variables and their initial values (these variables are used to predict the behavior of the model based on guard conditions and actions), “currentElementID” represents the unique identifier for the model element, “currentElementName” is the user-specified name for the model element,

“actions” indicate the change in the values of variables when a transition is taken through the element, and “properties” specifies extra information included in the model, i.e. a description regarding the node or edge.

Test Case Concretizer

To execute the test cases on Alstom’s test execution platform, we need to have concrete test cases, which contain the precise actions, data values, and signals’ names not present in the abstract test cases [2]. Our framework uses a mixed approach [9] for the concretization and execution of generated test scripts. The transformation of abstract test cases to concrete test cases is done while generating the test scripts, whereas the adaption layer is added at the execution level. The concretization of test cases includes the extraction of the test data from the actions (generated in a test step) from the JSON file and providing the system’s signals in an XML format, i.e. logical and technical signal names used by the TCMS, as shown in Fig. 3. Engineers at Alstom uses these logical signal names in requirements and test specifications at the initial levels of development and testing phases. Logical names are the initial names for signal representation, which are available to the developers and testers. However, these logical signal names must map to one or more technical signal names, where technical signal names are the actual signal names used by the SUT to execute operations. So, the last step in the concretization process includes rules to

```

<?xml version="1.0" encoding="UTF-8"?>
<Signals xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <Signal>
    <Type>Input</Type>
    <DataType>bool</DataType>
    <LogicalName>global.variable1</LogicalName>
    <PrimaryTechnicalName>XYZ</PrimaryTechnicalName>
    <SecondaryTechnicalName>XYZ</SecondaryTechnicalName>
    <Primary2TechnicalName> </Primary2TechnicalName>
    <Secondary2TechnicalName> </Secondary2TechnicalName>
  </Signal>
  <Signal>
    <Type>Output</Type>
    <DataType>bool</DataType>
    <LogicalName>global.variableoutputx</LogicalName>
    <PrimaryTechnicalName>XYZ</PrimaryTechnicalName>
    <SecondaryTechnicalName>XYZ</SecondaryTechnicalName>
    <Primary2TechnicalName>FYE</Primary2TechnicalName>
    <Secondary2TechnicalName>HUH</Secondary2TechnicalName>
  </Signal>
</Signals>

```

Fig. 3 Example of an XML file containing information about logical and technical names

associate these logical and technical signal names with test data generated in test cases.

Test Data Extractor

The test data extractor in the framework is responsible for reading the JSON file and extracting useful information from it such as model name, currentElement, currentElementID and actions containing the test data for test cases.

Signals' Information Extractor

The signals' information extractor in the framework is responsible for reading the XML file and extracting the information about each signal, such as the signal type (i.e. Input or Output), technical name(s) (i.e. primary and secondary technical names), and information related to the data type of its respective values.

Mapping Rules

The concretization of test cases is based on certain rules, which not only preserve the continuity of test cases and traversing order but also prevent information loss (Table 1). These rules work as a guideline for the framework to concretize the test cases and to provide a mapping between the test data (consisting of the logical name of the signals as per the requirement specification of the SUT) and the technical name of the signals (used by the SUT in the real world) along with their respective generated values.

Arrows between the elements of constructs in Table 1 denote the nested relationship between these elements, i.e., attributes grouped within other objects. For instance, the arrows between the elements of column 1 (Target Constructs) in Rule 1 denote that a 'Model' consists of 'actions', 'actions' contains 'Action', and 'Action' comprises multiple 'Strings'. To facilitate the mapping between logical and technical signals, we extracted the test data from each 'String' and used an intermediate 'list' consisting of 'VariableName and Value' to store it. In this context, Rule 1 defines a mapping between abstract test data extracted from the 'String' with a 'list' of variable names and their respective values in column 2 (Mapped Constructs). A mapping of each variable name from the 'list' with the 'logical names' of the signals (provided in the XML file) has been provided in Rule 2. Similarly, these 'logical names' are mapped to their corresponding 'type and technical names' of the signals in Rule 3 whereas Rule 4 provides a mapping between the 'values' (extracted from the JSON file) and the 'logical name' of the signals. Rule 5 maps the 'values of signal's logical names' with the 'values of respective technical names' of the signals. However, if a string contains a variable representing a timing constraint with the keyword 'Time', it has been mapped to the value of 'ResponseTime' in Rule 6, which will be used as a parameter in the verification step of the

Table 1 Mapping rules for test case concretization

ID	Target constructs	Mapped constructs
1	Model → actions → Action → Strings	List<VariableName, Value>
2	List<VariableName, Value> → VariableName	Signals → Signal → LogicalName
3	Signals → Signal → LogicalName	Signals → Signal → SignalType Signals → Signal → PrimaryTechnicalName Signals → Signal → Primary2TechnicalName Signals → Signal → SecondaryTechnicalName Signals → Signal → Secondary2TechnicalName
4	List<VariableName, Value> → Value	Signals → Signal → LogicalName → Value
5	Signals → Signal → LogicalName → Value	Signals → Signal → PrimaryTechnicalName → Value Signals → Signal → Primary2TechnicalName → Value Signals → Signal → SecondaryTechnicalName → Value Signals → Signal → Secondary2TechnicalName → Value
6	List<VariableName, Value> → Variable = 'Time' → Value	ResponseTime → Value

generated test script. It is important to mention here that after the extraction of test data from a String as provided in Rule 1, each rule follows the order as given in Table 1 for concretization.

Test Script Generator

The test script generator uses the mapping rules, the data extracted from the JSON formatted test cases, the XML file containing logical and technical signal names, and the implementation details of the test scripts (i.e. format, classes and methods to be executed on the target test execution platform) to generate executable test scripts. The generated test scripts contain two main steps for each test case: forcing the input signals and verifying the output signals within a specific time. Based on the signal type, the test script generator generates the scripts to force and verify the signals accordingly. An example of a generated test script is shown in Fig. 4 where line 1 of the code represents the method name, lines 2 and 3 are used to write the logs for actions of the test script, lines 4 and 5 are used to force the signals, line 6 writes the log for verification steps, and lines 7 and 8 are used to validate the output signals within a timing constraint.

The generated test scripts can be executed on the test simulation platform for TCMS. However, for execution, the simulation platform also requires some libraries (specifically designed for the TCMS developed at Alstom) and a configuration file containing information related to SUT, paths for storing test logs, etc. The test simulation platform, after executing the test scripts, generates test verdicts specifying the failed and passed test steps.

Study Design

This section provides an overview of the research methodology used to perform an experimental evaluation of the proposed framework along with the process of industrial survey planning and execution.

Objectives and Research Questions

The main objectives of this study are threefold. Firstly, developing the test script generation framework based on MBT principles for SiL testing of embedded software.

Secondly, the generated test scripts are validated to ensure their accuracy in reflecting the behavior and conditions outlined in the model. Lastly, to gain insights into the possible industrial adoption of MBT. Based on these main objectives, we have defined the following research questions:

- RQ1: How accurately do the TIGER-generated test scripts reflect the behaviour of a SUT, and to what extent can they be used to detect faults?
- RQ 2: To what extent can industries adopt and benefit from model-based testing methodologies, and what are the key factors influencing its successful adoption?

Experimental Design (RQ1)

We performed the evaluation of generated test scripts using an industrial case study of a fire detection sub-system controlled by the TCMS to address RQ1. Initially, we developed an EFSM model representing the SUT using requirements and test specifications. We have generated test cases and scripts with the TIGER and executed the test scripts on the SUT. We have also injected some faults in the model, and generated and executed test scripts to evaluate the framework in terms of fault detection from a conformance perspective.

SUT

The case selected for the evaluation belongs to an ongoing TCMS development project for the MOVIA vehicle product of Alstom's family of metro cars. TCMS is the center of a distributed system, built upon an open standard IP technology to communicate between and control multiple sub-systems (i.e. doors, air conditioning, ventilation, etc.) of the modern train. In discussions with Alstom, we have selected requirements of the SUT related to the fire detection sub-system of TCMS [10]. TCMS uses a fire detection system for the indication of two types of fires in the driver's cab: internal fire and external fire. The fire detection system uses two instances of a Fire Detection Control Unit (FDCU) device. Both FDCUs can have two states: slave and master. The fire detection system sends a signal to the TCMS about the status of fires depending on the state of each FDCU. Based on these signals, TCMS indicates internal or external fire.

Fig. 4 An Example of generated test steps in C# script

```
public override void TestSteps() {
    this.Trace(LogType.NewStep, "InternalFireInvalid");
    this.Trace(LogType.Action, "InternalFireInvalid");
    this.StepResult = this.cabSelected.CarPwrSupply.CarFireProtect.ForceSignalValue("XYZ", false);
    this.StepResult = this.cabSelected.CarPwrSupply.CarFireProtect.ForceSignalValue("FYE", false);
    this.Trace(LogType.Verification, "indicate");
    this.StepResult = this.cabSelected.CarPwrSupply.CarFireProtect.WaitForSignalCondition("FYZ", false, 2500);
    this.StepResult = this.cabSelected.CarPwrSupply.CarFireProtect.WaitForSignalCondition("XYE", false, 2500);
}
```


Modelling of SUT

We have modeled the expected behavior of the aforementioned sub-system in GW studio by exploring and understanding the requirements and test specifications, as well as by getting continuous input from the testing team at Alstom. GW studio is one of the versions of GW that create and validate the system's EFSM model. The model created in GW Studio consists of nodes, edges, and guard conditions. Nodes represent the states, edges represent the transition taken by the system from one state to another, and guard conditions are Boolean expressions representing the expected behavior of the model. To model the fire detection system, we have identified all possible states, transitions, and guard conditions of the system based on requirements and test specifications. Figure 5 depicts the EFSM model in three diagrams representing the SUT. The model explicitly representing the requirements of the fire detection system of TCMS consists of three diagrams, 14 states, 39 edges, and 15 data variables (i.e., 12 boolean inputs, two boolean outputs, and one integer representing timing constraint).

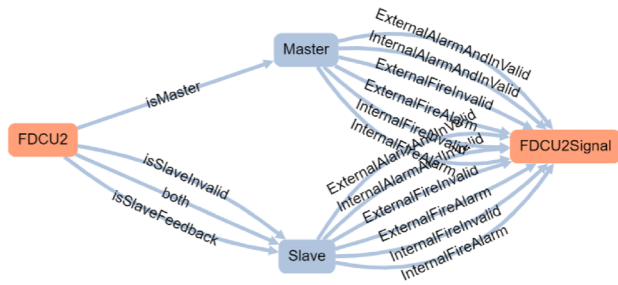
Two diagrams Fig. 5a and b represent the FDCUs and one diagram Fig. 5c represents the TCMS as a black box. FDCU1 is an initial as well as shared node of the model. FDCU1Signal, FDCU2, FDCU2Signal, TCMSisActive and FDCUsFireSignals are the rest of the shared nodes of the models. These shared nodes are used by GW to traverse between different models while validating the model and generating test cases. Master, Slave, InternalFire, ExternalFire, InternalAlarmAndInvalid and Reset nodes represent the other states of the SUT based on requirements and test specifications. Similarly, edges, data variables, and corresponding guard conditions were added based on the expected behavior of the SUT.

Fault Injection into EFSM Model

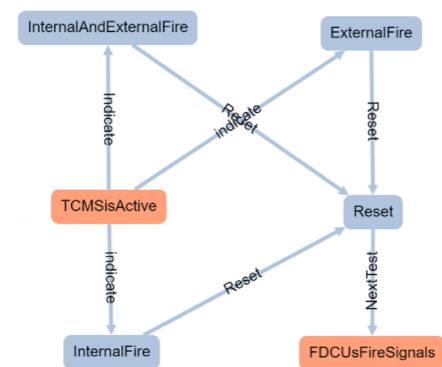
Due to the unavailability of source code, we have injected some faults based on the different mutant operators, which omit or insert model elements [7] to produce faulty test scripts. We have used three mutant operators (output exchanged, change in guards conditions, and state missing) and created three versions of the faulty model. In the first version of the faulty model, we exchanged the output values of internal and external fire such that if a system is supposed to indicate the internal fire, it will indicate the external fire and vice versa. Similarly, in the second version, we made some changes in the guard conditions while we removed the Master state of one of the FDCUs in the third version to make these models contradictory to the original specification.



(a) Diagram representing FDCU 1



(b) Diagram representing FDCU 2



(c) Diagram representing subsystem of TCMS

Fig. 5 EFSM model representing the fire detection system controlled by Alstom's TCMS

Survey Design (RQ2)

To gain industrial insights and estimate the potential of MBT for the testing of embedded systems at the system level, we conducted a survey by following three phases [34]: (1) survey planning and design (2) survey distribution and data collection (3) data analysis as shown in Fig. 6.

Survey Planning and Design

In the planning and design phase, we identified the main goal as defined in “Objectives and Research Questions” for conducting the survey and selected the target audience of industrial practitioners based on their direct or indirect affiliation with software testing activities of embedded software.

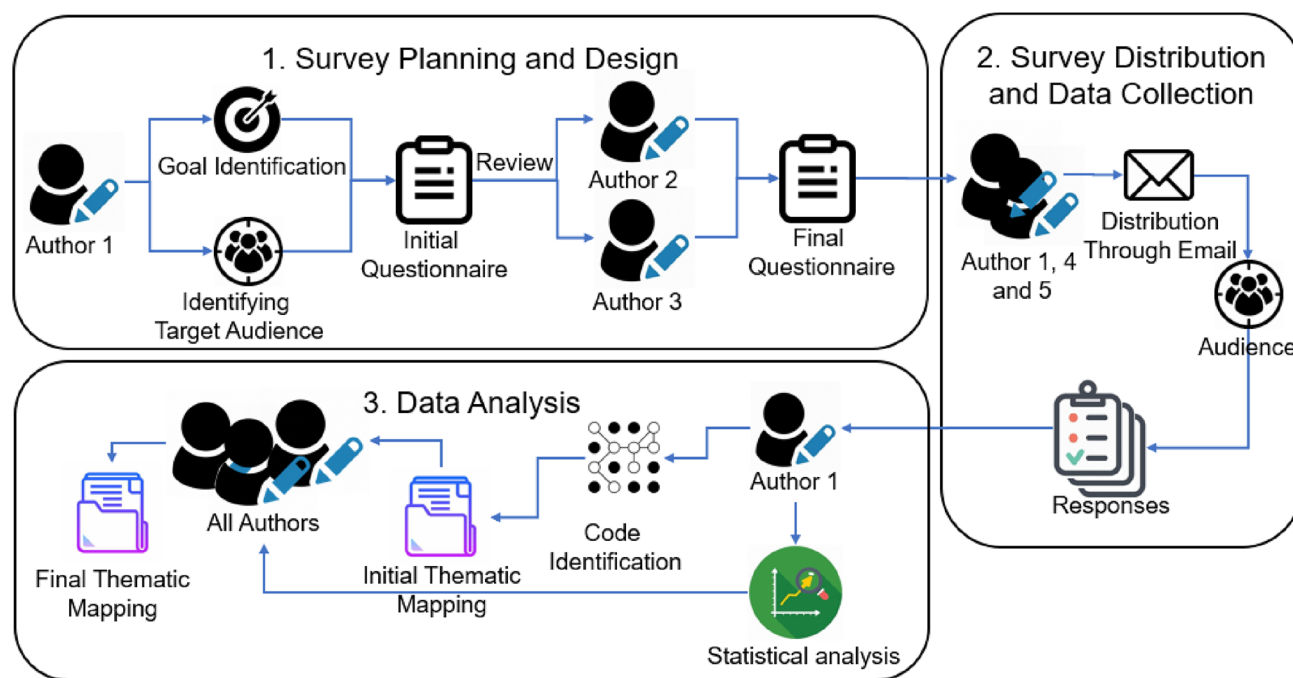


Fig. 6 An overview of survey planning, conduction, and data analysis

Based on defined goals and target audience, we created an anonymous online survey consisting of two sections and 16 questions in an iterative manner. Author 1 created the initial questionnaire, and authors 2 and 3 performed a review process to finalize the questionnaire. The first section inquires about the background of participants, i.e., their industrial role and its description, years of experience, and the type of testing they usually perform. The second section contains all questions relevant to the challenges, perception of MBT, and applicability of MBT in an industrial context. To collect both quantitative and qualitative data, we designed six closed-ended and ten open-ended questions. For closed-ended questions, checklists and rating scales (i.e., 1 for extremely low and 10 for extremely high) were provided, followed by an open-ended question, asking participants to specify the reasoning of their choice to gain better insights and cater to the problems associated with the random selection. The complete survey is annexed at the end of this paper as Appendix A. We can not publish the raw data to honour our commitment to the confidentiality of respondents' data.

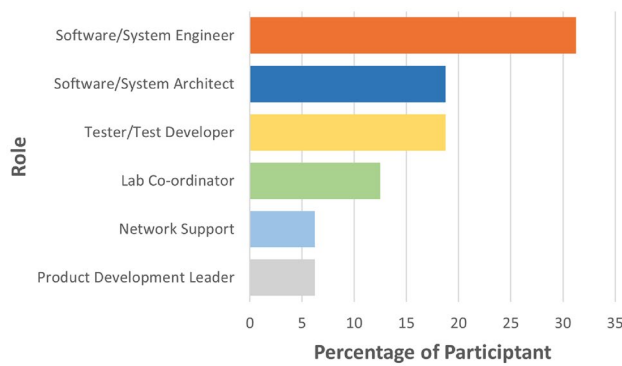
Survey Distribution and Data Collection

We disseminated the survey among 57 participants at Alstom through email. Authors M. N. Zafar, Z. Haider, and I. Singh were responsible for the distribution of the survey to the targeted audience at Alstom. All the participants were advised to complete the survey within a predetermined timeframe. We also encouraged them to provide detailed and

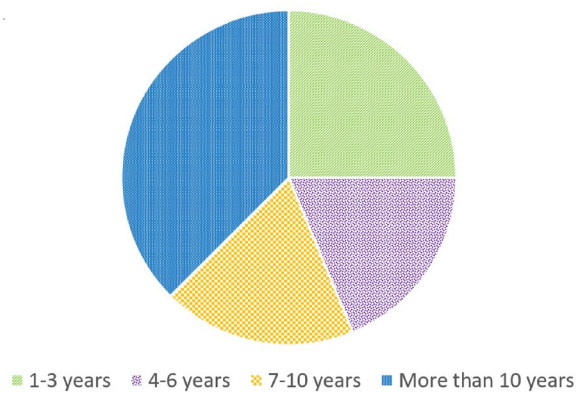
honest responses to ensure the accuracy and reliability of the data. Moreover, we have performed rigorous quality checks such as removing duplicate responses, ensuring respondent anonymity, timeframe validation, etc. to uphold the integrity of the data.

Exactly 16 participants responded to the survey by the closing date, suggesting a 28% response rate. We have encountered different challenges in getting the responses to the survey from industrial practitioners including time constraints and their priorities towards completing their assigned industrial tasks. However, to increase the number of responses, we extended the closing date and time of the survey, allowing participants additional time to respond, and dispatched gentle email reminders to the targeted audience with follow-up messages. Figure 7a and b show the industrial role and number of years of experience of the respondents, respectively. As shown in Fig. 7a, 31.25% of the respondents were working as software/system engineers, 18.75% each was associated with the role of software/system architects and Tester, 12.5% were lab coordinators and 6.25% each had the responsibility as product development leaders and network support. Similarly, 37.5% of the respondents had experience of more than 10 years, 18.75% each lies in the range of 4–6 and 7–10 years, and 25% were affiliated with the industry between the duration of 1–3 years, as presented in Fig. 7b.

For the demographic significance, 56.25% of the respondents were directly or indirectly involved in behavioral testing, 12.5% of respondents were associated with



(a) Percentage of respondents and their role



(b) Experience of respondents in years

Fig. 7 Demographic profile of respondents

structural testing, and 31.25% of respondents were affiliated with both types of testing of embedded software.

Data Analysis

We performed a thorough qualitative and quantitative analysis of the collected data. The quantitative data was analyzed using statistical analysis and by creating different graphs (i.e., scatter plots, bar plots, and pie charts). For qualitative data, we applied a thematic analysis approach by following Cruzes et al.'s [15] recommended steps. Author M. N. Zafar performed the initial reading of the responses to get familiar with the data, conducted the statistical analysis, identified the codes or specific segments from the text, translated the identified codes into sub-themes, and created higher-order themes using recommended mapping steps. After the creation of the initial thematic map and statistical analysis, all the authors reviewed and refined the thematic map to compile the results as reported in “RQ2: Industrial Insight Through Survey”.

Table 2 Comparison between TIGER-generated test scripts using the correct model and manually-written test scripts

Test generation source	No. of test steps (min–max)	No. of failed test steps
Correct model	264–514	0
Manually written	24	0

Results

RQ1: Experimental Results

This section describes our findings based on fault injection analysis to evaluate the test scripts generated by TIGER.

Generation and Execution of Test Scripts

After creating different versions of the model, we generated the test cases and scripts multiple times using each model. It is important to mention here that to ensure comprehensive and effective test coverage, we used a random generation algorithm and 100% edge_coverage as stopping condition offered by GW to generate abstract test cases guaranteeing that all functional paths, states, and edges in the model are exercised. All the other applicable generation strategies, except for the quick_random algorithm with 100% edge coverage, which produced similar results, resulted in fewer test cases and only partial model coverage, offering no significant value to our research objectives. We also provided logical and technical names of the signals to TIGER and executed the test scripts on the SUT. Here, we report results from five test generations based on the variation in the number of generated test steps and the failed test steps. Moreover, to validate the accuracy and fault detection capability, we have also comparatively analyzed the verdict produced by TIGER-generated test scripts and test scripts written manually by a tester at Alstom.

Fault Injection Analysis

As shown in Table 2, no fault was identified in the SUT by executing the manually written test scripts. The correct version of the model conforms with the requirements specification of the SUT, hence no failed test steps were reported on each execution of test scripts generated using it. On the other hand, a different number of failed test steps were identified by the test scripts generated from faulty models (as shown in Table 3). However, the number of generated test steps was different in each test script due to random walks of GW for the generation of test cases. We have also analyzed the generated test scripts to evaluate their requirements' coverage. We observed that all test scripts generated using the correct

version of the model contained at least one test case for each requirement and provided 100% requirements coverage. It was also observed that one of the test scripts generated using the ‘change guard’ mutant model missed the combination required to produce failed test steps. Similarly, no failed test step(s) were identified in some of the test scripts with a ‘state missing’ mutant. It was attributed to the missing *Master* state and GW made the FDCU ‘slave’ in initial steps while generating the test cases, hence providing no combination that could produce a failed test step.

In summary, the results show that when faulty models did not conform to requirements, TIGER-generated test scripts produced failed test steps on execution. However, there were two exceptions where GW did not generate the required failure-triggering combinations. In comparison, the test scripts generated using the correct version of the model discovered no fault in the SUT due to their conformance with the requirements specification and the implemented SUT. Hence, these results showcase the efficacy and accuracy of the generated test scripts to represent real-world scenarios defined by a model as well as their abilities and potential weaknesses to detect faults, therefore reinforcing the reliability of the TIGER.

RQ2: Industrial Insight Through Survey

This section presents the findings of the conducted survey to provide the answer to RQ2. We have divided the compiled results and themes into two discussion topics as follows:

- **Industrial challenges in testing and expectations from MBT:** It discusses the testing challenges reported by industrial practitioners in relation to the potential benefits MBT can offer.
- **Industrial adoption of MBT and its challenges:** It delves into the perception of MBT among industrial practitioners and the factors that can influence its adoption in an industrial context.

A detailed discussion on these topics in the context of collected responses is provided below. Moreover, to substantiate the connections between the themes and the data, we have presented the quotes from the survey in distinct boxes.

Industrial Challenges in Testing and Expectations from MBT

This topic deals with one of the important aspects of our conducted survey, which is to determine the relationship between the benefits of MBT and the challenges of embedded software testing in the industrial context. The themes relevant to the topic are reported in Table 4.

Theme 1: Industrial testing practices and its challenges. Figure 8 represents the challenges experienced by

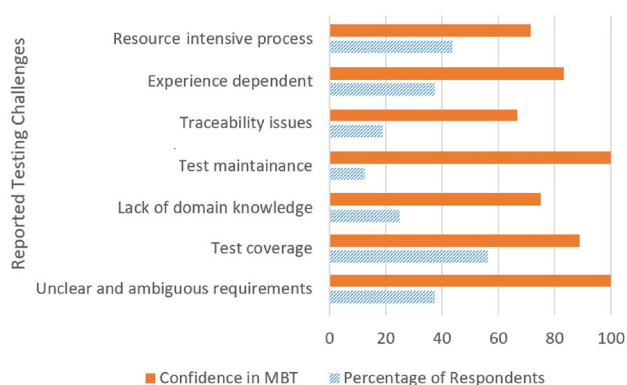
Table 3 Comparison between TIGER-generated test scripts using correct and faulty models

Test Generation Source	Test Generation #	Test Generation 1			Test Generation 2			Test Generation 3			Test Generation 4			Test Generation 5		
		No. of Test Steps	No. of Failed Test Steps	of	No. of Test Steps	No. of Failed Test Steps	of	No. of Test Steps	No. of Failed Test Steps	of	No. of Test Steps	No. of Failed Test Steps	of	No. of Test Steps	No. of Failed Test Steps	of
Faulty Model (1)	Output Exchange	329	28		364	24		349	28		544	37		179	19	
Faulty Model (2)	Change Guard	624	6		239	0		133	1		514	2		254	4	
Faulty Model (3)	State Missing	270	1		399	1		187	3		248	0		365	2	
Correct Model	NA	264	0		454	0		514	0		294	0		299	0	

Red shade shows failed test steps and green shows no failures detected

Table 4 Themes and sub-themes relevant to industrial challenges in testing and expectations from MBT

Theme	Sub-theme
1. Industrial testing practices and its challenges	1.1 Manual industrial testing is resource-intensive 1.2 Experience-dependent process 1.3 Lack of test maintenance and traceability between requirements and test cases 1.4 Lack of domain knowledge and test coverage 1.5 Unclear and ambiguous requirements
2. Relationship between industrial challenges and MBT benefits	2.1 Resource efficiency to generate test cases through a model conforming to requirements 2.2 Generation of effective test cases in terms of coverage 2.3 Improving understandability of documented requirements through modeling

**Fig. 8** Reported challenges in testing and confidence in MBT to resolve the reported challenges

industrial practitioners, along with the level of confidence they have in MBT to address the stated challenges. The analysis of data reveals a lack of assurance of comprehensive test coverage as the main challenge in industrial practices, reported by a portion of 56.25% of the respondents. Some of them also aligned it with the challenge of interpreting ambiguous and unclear requirements, a lack of domain knowledge, and the experience of a tester. Around 50% of the respondents highlighted the challenge of managing resources such as time and cost associated with testing activities. Whereas, issues of traceability between requirements and test cases, and test maintenance are reported by 18.75% and 12.5% of the respondents, respectively.

“Most of the testing is done manually, and the selection of test cases depends on the tester’s experience and skills to prioritize the tests. This creates a main problem of missing important functional tests, and doing them manually is time-consuming.”

Theme 2: Relationship between industrial challenges and MBT benefits. To assess the potential of MBT’s ability to address industrial challenges, the survey also examines the opinion of industrial practitioners concerning their reported challenges. As shown in Fig. 8, we deduced general positive

expectations from the MBT. Most of the respondents believe that different MBT activities can be used to address these challenges. For instance, a majority of respondents identified modeling and models representing the system as a powerful tool that can be used to eradicate the problem of unclear/ambiguous requirements and to gain an understanding of the system and interactions between subsystems. Similarly, they also believe that the automatic generation of test cases from models conforming to the system requirements and using different coverage criteria can assist in covering all relevant scenarios of the system. Several respondents also indicated the benefits of automation using MBT for the effective use of resources but also showed their concerns regarding the time needed to create a model and the cost of training and learning modeling languages and tools.

“My experience is that text-based descriptions have a wider range of interpretation compared to models/drawings. Thus, the chance of misalignment throughout development between concept/requirement compared to implementation and testing is greater.”, and “MBT can be employed to address all these challenges due to its inherent nature of relying mainly on models. Certainly, those design/implementation models can be utilized to automatically generate test cases and contribute to time-efficient and effective test cases. Secondly, models are indeed superior descriptions of the systems in comparison to textual representation.”

Industrial Adoption of MBT and Its Challenges

For the successful implementation and utilization of MBT in the industry, it is crucial to investigate the contributing factors influencing MBT adoption. For this reason, we included multiple questions targeting the perception of MBT and modeling notation among industrial practitioners, limitations, and integration of MBT tools with existing processes. Table 5 contains themes related to the topic.

Theme 3: Perception of adopting MBT and Finite State Machine (FSM) as modeling notation in the

Table 5 Themes and sub-themes relevant to the industrial adoption of MBT and its challenges

Theme	Sub-theme
3. Perception of adopting MBT and Finite State Machine (FSM) as modeling notation in the industry	3.1 Modeling is a time-consuming process that also requires extensive knowledge about the domain and modeling language 3.2 MBT models can be easily reused and maintained 3.3 Highly complex to create an FSM model, especially when a stateless function is involved 3.4 MBT is adoptable at the system level 3.5 MBT can be seamlessly integrated with the existing in-house tools
4. Challenges associated with MBT, and its industrial adoption	4.1 Challenges of reliability and validation of MBT models and tools 4.2 Lack of awareness of MBT practices, methodologies, and tools 4.3 Resource constraints

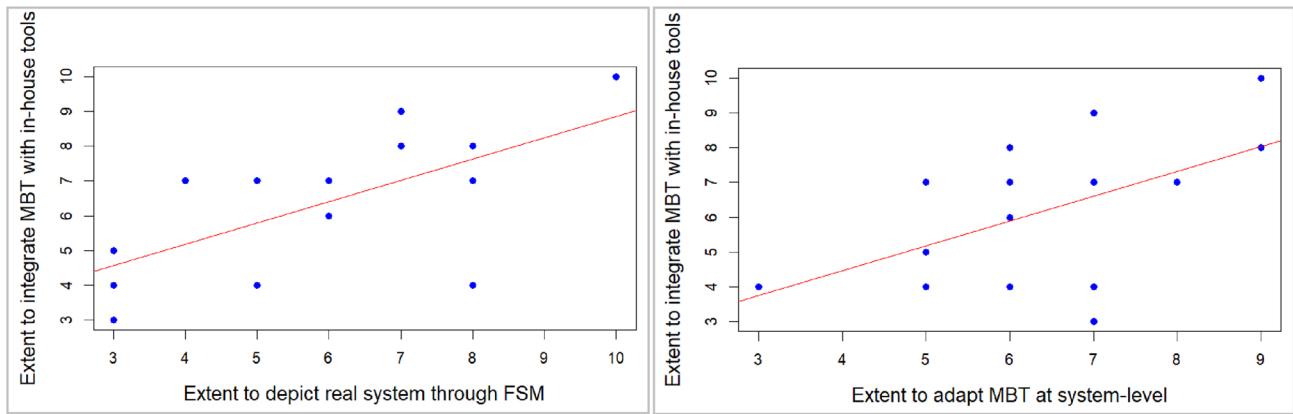
industry. To analyze the adoption of MBT and FSM as the modeling notation, we included different close-ended questions targeting the ability of a The FSM-based model is used to depict the real behaviour of an embedded system, adopt it at the system level, and integrate MBT with existing in-house tools. We suggested that the participants choose ratings that reflect their level of satisfaction. The minimum rating indicates the least favourable choice, whereas the maximum rating indicates the most favourable choice of the respondents. The minimum ratings provided by the respondents against the above-mentioned topics is 3 for each. On the other hand, the maximum ratings provided by the respondents are 10, 9, and 10, respectively. However, in general, all the respondents provided an average of 5.75, 6.50, and 6.25 ratings, respectively. Figure 9a shows the scatter plots and Fig. 9b presents a 3D-correlation plot of the rating provided by the respondents. The scatter plots in Fig. 9a show a generally positive outlook on the integration of MBT with in-house tools and the two factors: the extent to which FSM notation depicts the system behavior and the extent to which MBT adapts to testing at the system level. This positive outlook is an indication that a reasonable percentage of respondents agree that a tool-supported transition to MBT is feasible. The 3D-correlation plot in Fig. 9b quantifies this positive outlook with correlation coefficient calculations.

Moreover, we also analyzed the reasons specified by the respondents for their chosen scales. As mentioned above, most industrial practitioners have positive expectations from MBT, however, and also think that modeling an embedded system is a complex task that also requires resources and knowledge about the domain and the modeling language (i.e. FSM). We found a mixed perception regarding the understandability of an FSM-based model and the accuracy of FSM as a modeling language to represent the actual behavior of the system. Around 50% of the respondents reported it as a convenient and conventional method of modeling to understand the requirements of a system, 25% of the respondents found it complex, whereas 25% of the respondents indicated

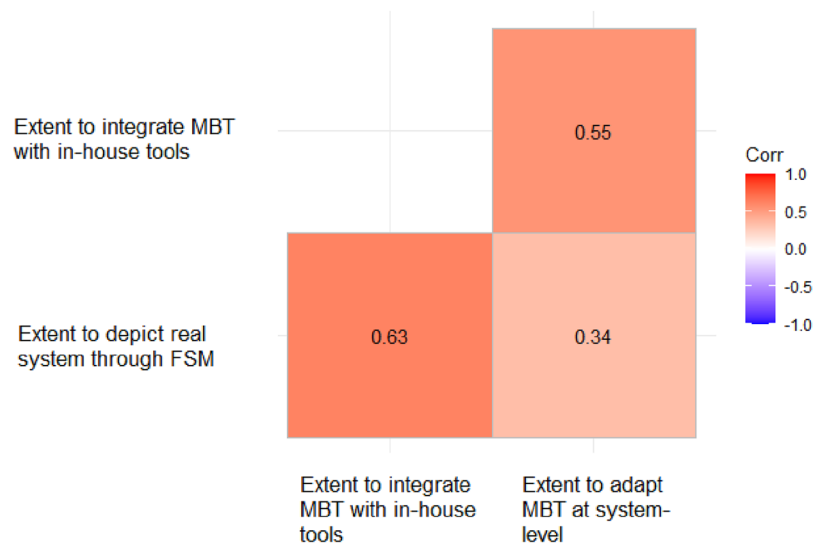
a neutral stance. They also identified that the complexity of modeling a system through FSM increases with the increase of requirements of a system, especially when a stateless function is involved. A few respondents also believe that the models of the system can also provide benefits in terms of reusability and maintainability across the projects while reducing the cost of the development life cycle. The qualitative data analysis also reveals that some respondents also provided a low rating against FSM or MBT adoption due to their limited or basic knowledge of MBT or lacking practical experience with MBT tools.

“State machine is a behavioral model and is a commonly known diagram to define the behavior of systems. It should not be very challenging to understand the flow and acquisition of MBT.”, and “FSM is not the best notation to capture stateless logic and sequenced scenarios. It creates too complex models in many cases.”

Theme 4: Challenges associated with MBT, and its industrial adoption. The analysis of data reveals three main challenges associated with the industrial adoption of the MBT: ensuring the reliability of the MBT tools and models, lack of awareness and knowledge about MBT practices, and lack of resources. We found the reliability and validation of MBT processes and tools as a major limitation to its industrial adoption. To overcome this challenge, respondents also suggested establishing the trust of industrial practitioners and organizations in the reliability of MBT tools and processes through conducting validation activities in industrial settings. Based on our analysis, we also identified that most organizations are reluctant to invest in MBT due to the cost associated with training and customization of industrial tool-chains, fear of altering well-established industrial practices and existing workflows, and concern regarding return on investment. Moreover, several respondents also highlighted that the successful implementation and utilization of MBT approaches in the industry requires awareness of MBT tools,



(a) Scatter plots of the ratings provided by respondents on the integration of MBT with in-house tools and two other factors: the extent of an FSM depicting system behavior and the extent of MBT adaptation at system-level testing



(b) A 3D-correlation plot of the ratings provided by respondents

Fig. 9 Scatter plots and a 3D-correlation plot for integration of MBT with in-house tools and the two other factors

modeling languages, MBT workflow, and a collaborative environment with MBT experts.

“You need domain and MBSE experts to be involved and collaborating through all adoption phases i.e., planning and assessment of the model and tools, early adoption/piloting, and deployment/launch in the enterprise.”, and “(...) The company would have to commit large resources to such a shift, to ensure ongoing deliveries are not threatened, yet enough money to ensure success. (...)”

Discussion

We utilized GW for abstract test generation due to it being open-source, integration capabilities, extensive documentation, powerful generation algorithms and coverage criteria. There are various studies [11, 35–37] that have also shown the effectiveness of MBT, by integrating GW with other MBT tools (e.g., CANoe⁺ and UPPAL) and testing frameworks (e.g., Selenium). However, we found only one relevant paper [38] that utilizes GW for the unit-level testing of *embedded systems* with a high-level description

of the implementation to concretize abstract test cases. Our work targets to fill the research gap in system-level model-based testing of embedded software where we provide significant details on the concretization process to generate executable test scripts. Moreover, validation of the proposed framework in an industrial setting showcases its potential industrial impact.

For validation purposes, we have used three different mutation operators to evaluate the TIGER-generated, however, we also tried other mutation operators discussed in [7]. These other mutation operators did not result in any different behavior in our case. For example, we induced some faults based on ‘arc missing’, ‘output missing’, ‘event missing’, ‘destination exchanged’, and ‘event exchange’ operators. However, event and destination exchange had a similar effect on the model as output exchange. Similarly, output missing, event missing, and arc missing showed no effect on the model, and induced faults based on these operators only resulted in fewer input combinations with no failed test steps, so we have neglected these operators in our study.

The survey results show a generally positive industrial viewpoint on MBT capabilities with respect to overcoming existing testing challenges. However, modeling the system under test was regarded as a time-consuming process, that also requires extensive domain knowledge and knowledge about the modeling language. Thus, research into automating the model generation from existing artifacts is an interesting area of research. The survey also highlights factors limiting the adoption of MBT, and two of them relate to a lack of awareness and required resources for training. We believe these two factors can be overcome to a certain extent by working towards integrating MBT within in-house tools, which our survey results have identified as feasible.

Due to TIGER-related training costs required by industrial practitioners and for not diverting them from their routine tasks, we did not mention TIGER in the industrial survey. Also, our target was to keep the survey topic general and not tool specific. Yet, the survey results indirectly identify opportunities to further develop TIGER and to improve its practical relevance. For instance, the analysis indicates that FSM models can also be used to address the problems of ambiguous requirements, lack of domain knowledge, and comprehensive test coverage at the system level. However, it also suggests training industrial practitioners with modeling notations (i.e., FSM) and the complexity of modeling a system with stateless functions. Similarly, multiple respondents also highlighted the importance of effective resource utilization through MBT, indicating the need to minimize the number of TIGER-generated test cases while preserving its fault detection effectiveness. This is something that we have started investigating already [39].

Validity Threats

We generated the test cases using an abstract model representing the behavior of the SUT and ensuring the correctness of the model contributes to one of the potential threats to the internal validity of this study. It took us several rounds of modeling to completely understand the requirements and the test specifications to arrive at a correct model that an Alstom test engineer eventually confirmed as correct. The other internal validity threats are related to the selection bias of respondents and the random selection of closed-ended questions due to different interpretations of the questions or attitudes toward MBT. We mitigated these threats by targeting a diverse audience with a particular interest in software testing, asking the participants to specify the reason to justify their selection, and performing the thematic analysis based on guidelines provided in [15].

The threats to external validity and reliability related to the evaluation of the proposed framework include human experience, modeling notations, and generator algorithms. We expect that if a person with similar modeling and testing experience will replicate this study using random walk and edge coverage criterion of GW, similar results should be achieved. However, different modeling notations and generator algorithms may produce different results. Another issue is that the framework is specifically designed for CPS testing at Alstom, so it has particularities that may not be applicable to other CPSs but still be applicable to multiple projects inside Alstom. Nevertheless, the description of the framework and the mapping procedure can give clues to companies operating in similar domains to apply MBT in practice. Moreover, another threat to the reliability of this study is the lower number of mutants. We created distinct mutants at the model level based on mutation operators, however, due to the abstract nature of the model, various mutants either provided an overlap in the type of faults they introduced (as discussed in “Discussion”) or failed to represent realistic faults. Hence, we carefully analyzed and selected each mutant to reflect significant changes and distinct issues in the generated test suite. We may also want the mutation testing at the model level to be supplemented with lower, code-level mutations and then validate our framework. We did not have access to code for this study, but this research direction is worth investigating if it becomes possible. Other potential threats to the external validity and reliability of the survey include sampling bias, sample size, low response rate, and contextual factors. Although the survey targets the industrial insight to adopt MBT, it lacks results from other organizations and domains. Similarly, the lower response rate than the desired poses a threat to its generalizability while

introducing non-response bias. Moreover, the diversity in regulatory and technological frameworks across industrial domains may affect the interpretation of MBT as well as produce different results.

Conclusion and Future Work

We have proposed an MBT framework, TIGER, focused on the concretization of abstract test cases and the generation of test scripts for CPSs where embedded software plays an important part. There are three main parts of TIGER: abstract test case generator, test case concretizer, and finally, test script generator. We have evaluated the capabilities of TIGER-generated test scripts to represent real-world scenarios and detect faults using model-based mutation analysis. Moreover, we have also conducted an industrial survey to gain an understanding of industrial insight into the real-world adoption of MBT and associated challenges. The experimental results show that test scripts generated by TIGER are executable, contain concrete test data, and can be used to uncover interaction faults at the SiL simulation level. The test scripts generated through the correct model did not result in any failed execution step, confirming the correct generation and execution, and ensuring conformance to the requirements specifications and the implemented SUT. Furthermore, the qualitative and quantitative analysis of the conducted survey shows that MBT can address most of the identified industrial challenges in testing. The practitioners also agree that MBT tools can be integrated with existing in-house tools and can be adopted at the system level for the generation of test cases for embedded software. However, a lack of resources, a lack of awareness about MBT tools and methodologies among industrial practitioners, and ensuring the reliability and validation of MBT tools and processes were found to be the main contributing factors to its limited adoption.

For a thorough evaluation of TIGER, a proper cost-benefit analysis is required. Although, a large number of generated test steps from TIGER yielded greater execution time than manually written test steps, an added benefit may be better combinatorial coverage than manually-written test scripts. However, a proper quantitative comparison with combinatorial testing is left as an interesting future work, along with the optimization to reduce the generation time and configuration for use at a different level of simulation testing (application level) for testing Alstom's CPS. Moreover, we also see the potential of conducting follow-up industrial surveys by targeting industrial practitioners from diverse organizations and domains.

Appendix A: Survey Questionnaire

Section 1: Demographic

1. What is your role and its description at your company? (Open-ended)
2. How many years of testing and development experience do you have? (Closed-ended)
3. What type of testing (i.e. behavioral, structural, etc.) do you usually perform? (Open-ended)

Section 2: About Experience and Expectations with Model-Based Testing

4. According to your experience, what are the main issues/problems you in general experience while manually designing the tests and performing the testing of safety-critical software? (Open-ended)
5. To what extent, do you think the model-based testing approach will address your specified testing problems? (Open-ended)
6. What expectations do you have from a model-based testing approach as a tester compared to your existing way of working? (Closed-ended)
7. Please specify your reasoning for choosing the answer(s) to Question 6. (Open-ended)
8. What other main benefits can you think of in adopting the model-based testing approach? (Open-ended)
9. To what extent do you think it is difficult to understand the state machine modeling notation using model-based testing? (Closed-ended)
10. Please specify the reasoning for choosing the scale in Question 9. (Open-ended)
11. To what extent do you think using Finite State Machine (FSM) as a modeling notation in the model-based testing approach can depict the real behavior of the system using models? (Closed-ended)
12. Please specify the reasoning for choosing the scale in Question 11. (Open-ended)
13. To what extent can the model-based testing approach be adopted and used for system-level testing at your company? (Closed-ended)
14. To what extent do you think the model-based testing approach and tool will fit with the existing in-house tools at your company? (Closed-ended)
15. According to your experience, what expertise do you think is required at the organizational level to adopt the model-based testing approach? (Open-ended)
16. What are the main limitations you can think of in adopting the model-based testing approach? (Open-ended)

Acknowledgements This research work has received funding through the AIDOaRt project from the ECSEL Joint Undertaking (JU) under Grant agreement no. 101007350 and the SmartDelta (ITEA) [40] project.

Author Contributions MNZ: conceptualization, methodology, software, investigation, writing—original draft; WA: conceptualisation, resources, writing: review and editing, project administration, and funding acquisition; EPE: conceptualisation, writing: review and editing; ZH: validation, resources, writing: review; IS: validation, resources, writing: review.

Funding Open access funding provided by Mälardalen University. This research work has received funding through the AIDOaRt project from the ECSEL Joint Undertaking (JU) under grant agreement No 101007350 and the SmartDelta (ITEA) [40] project.

Availability of Data and Materials The raw data collected from the study participants is not available publicly. TIGER is available at <https://github.com/MuhammadNoumanZafar/TestScriptGeneration.git> based on GraphWalker (GW) <https://github.com/GraphWalker/graph-walker-project/wiki>.

Declarations

Conflict of Interest On behalf of all authors, the corresponding author states that there is no Conflict of interest.

Ethical Standard Informed Consent The following statement was added to the survey: “Your information is highly confidential, the following information is for internal research purposes only”. Moreover, the industrial contact persons in the author list have read and approved the manuscript for publishing.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Binder RV, Legeard B, Kramer A. Model-based testing: where does it stand? *Commun ACM*. 2015;58(2):52–6.
- Kramer A, Legeard B. Model-based testing essentials-guide to the ISTQB certified model-based tester: foundation level. John Wiley & Sons; 2016 Apr 11.
- Xu D, Xu W, Kent M, Thomas L, Wang L. An automated test generation technique for software quality assurance. *IEEE Trans Reliab*. 2014;64(1):247–68.
- Paiva ACR, Maciel D, da Silva AR. From requirements to automated acceptance tests with the RSL language. In: International conference on evaluation of novel approaches to SE. Berlin: Springer; 2019.
- Kholkar D, Yelure P, Tiwari H, Deshpande A, Shetye A. Experience with industrial adoption of business process models for user acceptance testing. In: Modelling foundations and applications: 9th European conference, ECMFA 2013, Montpellier, France, July 1–5, 2013. Proceedings 9. Berlin: Springer; 2013. pp. 192–206.
- Alégroth E, Karl K, Rosshagen H, Helmfridsson T, Olsson N. Practitioners' best practices to adopt, use or abandon model-based testing with graphical models for software-intensive systems. *Empir Softw Eng*. 2022;27(5):103.
- Belli F, Budnik CJ, Hollmann A, Tuğlular T, Wong WE. Model-based mutation testing—approach and case studies. *Sci Comput Program*. 2016;120:25–48.
- Zafar MN, Afzal W, Enouï EP, Stratis A, Sellin O. A model-based test script generation framework for embedded software. In: International Conference on Software Testing, Verification and Validation Workshops (ICSTW); 2021, Porto de Galinhas, Brazil: IEEE, pp. 192–198.
- Utting M, Legeard B. Practical model-based testing: a tools approach. Amsterdam: Elsevier; 2010.
- Zafar MN, Afzal W, Enouï EP, Stratis A, Arrieta A, Sagardui G. Model-based testing in practice: an industrial case study using GraphWalker. In: 14th Innovations in software engineering conference (formerly known as India software engineering conference), 2021, New York, NY, USA: ACM; 2021, pp 1–11.
- Garousi V, Keleş AB, Balaman Y, Güler ZÖ, Arcuri A. Model-based testing in practice: an experience report from the web applications domain. *J Syst Softw*. 2021;180: 111032.
- Krenn W, Schlick R, Tiran S, Aichernig B, Jobstl E, Brandl H. Momut: UML model-based mutation testing for UML. In: 2015 IEEE 8th international conference on software testing, verification and validation (ICST), Graz, Austria, IEEE; 2015. pp. 1–8.
- Aichernig BK, Jobstl E. Efficient refinement checking for model-based mutation testing. In: 2012 12th International conference on quality software. Xi'an, China, IEEE; 2012. pp. 21–30.
- Graue C. Qualitative data analysis. *Int J Sales Retail Mark*. 2015;4(9):5–14.
- Cruzes DS, Dybå T. Recommended steps for thematic synthesis in software engineering. In: 2011 International symposium on empirical software engineering and measurement. Banff, AB, Canada, IEEE; 2011. pp. 275–84.
- Mohacsi S, Felderer M, Beer A. A case study on the efficiency of model-based testing at the European space agency. In: 8th International conference on software testing, verification and validation. Graz, Austria, IEEE; 2015, pp 1–2.
- Junyi Wang, Xiaoying Bai, Linyi Li, Zhicheng Ji, and Haoran Ma. A model-based framework for cloud api testing. In *41st Annual Computer Software and Applications Conference*, volume 2. Turin, Italy, IEEE, 2017, pp 60–65.
- Marinescu R, Saadatmand M, Bucaioni A, Seceleanu C, Pettersson P. A model-based testing framework for automotive embedded systems. In: 40th EUROMICRO conference on software engineering and advanced applications. Verona, Italy, IEEE; 2014, pp 38–47.
- Anbunathan R, Basu A. Automation framework for test script generation for android mobile. In: 2nd IEEE international conference on recent trends in electronics, information and communication technology. Bangalore, India, IEEE; 2017, pp 1914–1918.
- Gupta P, Surve P. Model based approach to assist test case creation, execution, and maintenance for test automation. In: In Proceedings of the First International Workshop on End-to-End Test Script Engineering (ETSE '11); 2011, New York, NY, USA: ACM, 1–7.
- de Moura JL, Charão AS, Lima JCD, de Oliveira Stein B. Test case generation from BPMN models for automated testing of web-based BPM applications. In: 17th International conference

- on computational science and its applications. Trieste, Italy, IEEE; 2017, pp 1–7.
22. Bonfanti S, Gargantini A, Mashkoor A. Generation of behavior-driven development C++ tests from abstract state machine scenarios. In: International conference on model and data engineering. Berlin: Springer; 2018. pp. 146–52.
 23. Vanhecke J, Devroey X, Perrouin G. AbsCon: a test concretizer for model-based testing. In: 2019 IEEE international conference on software testing, verification and validation workshops (ICSTW). Xi'an, China, IEEE; 2019. pp. 15–22.
 24. Fragal VH, Simao A, Endo AT, Mousavi MR. Reducing the concretization effort in FSM-based testing of software product lines. In: 2017 IEEE international conference on software testing, verification and validation workshops (ICSTW). Tokyo, Japan, IEEE; 2017. pp. 329–36.
 25. Kriebel S, Markthaler M, Salman KS, Greifenberg T, Hillemacher S, Rumpe B, Schulze C, Wortmann A, Orth P, Richenhagen J. Improving model-based testing in automotive software engineering. In: Proceedings of the 40th international conference on software engineering: software engineering in practice; 2018, Gothenburg, Sweden: IEEE/ACM, pp. 172–80.
 26. Neto ACD, Subramanyan R, Vieira M, Travassos GH. A survey on model-based testing approaches: a systematic review. In: Proceedings of the 1st ACM international workshop on empirical assessment of software engineering languages and technologies: held in conjunction with the 22nd IEEE/ACM international conference on automated software engineering (ASE), Atlanta, Georgia, 2007, New York, NY, USA. Association for Computing Machinery; 2007.
 27. Shafique M, Labiche Y. A systematic review of state-based test tools. *Int J Softw Tools Technol Transf.* 2015;17(1):59–76.
 28. Utting M, Legeard B, Bouquet F, Fournier E, Peureux F, Veronotte A. Chapter two—recent advances in model-based testing. *Advances in computers*, vol. 101. Amsterdam: Elsevier; 2016. pp. 53–120.
 29. Marinescu R, Seculeanu C, Guen HL, Pettersson P. Chapter three—a research overview of tool-supported model-based testing of requirements-based designs. *Advances in computers*, vol. 98. Amsterdam: Elsevier; 2015. pp. 89–140.
 30. Li W, Gall FL, Spaseski N. A survey on model-based testing tools for test case generation. In: Itsykson V, Scedrov A, Zakharov V, editors. *Tools and methods of program analysis*. Cham: Springer International Publishing; 2018. p. 77–89.
 31. Claessen K, Hughes J. Quickcheck: a lightweight tool for random testing of Haskell programs. In: Proceedings of the fifth ACM SIGPLAN international conference on functional programming; Montréal, Canada, 2000, New York, NY, USA: ACM, pp. 268–79.
 32. Fellner A, Krenn W, Schlick R, Tarrach T, Weissenbacher G. Model-based, mutation-driven test-case generation via heuristic-guided branching search. *ACM Trans Embed Comput Syst.* 2019;18(1):1–28.
 33. Aichernig BK, Schumi R. Property-based testing of web services by deriving properties from business-rule models. *Softw Syst Model.* 2019;18:889–911.
 34. Molléri JS, Petersen K, Mendes E. Survey guidelines in software engineering: an annotated review. In: Proceedings of the 10th ACM/IEEE international symposium on empirical software engineering and measurement; 2016, New York, NY, USA: ACM, pp. 1–6.
 35. Tiwari S, Iyer K, Enou EP. Combining model-based testing and automated analysis of behavioural models using graphwalker and UPPAAL. In: 2022 29th Asia-Pacific software engineering conference (APSEC). Japan, IEEE; 2022. pp. 452–6.
 36. Gao Z. Comparative study on model based test of automotive automatic control system. In: 2023 IEEE 2nd international conference on electrical engineering, big data and algorithms (EEBDA), Changchun, China, IEEE; 2023. pp. 1628–32.
 37. de Castro-Cabrera MC, García-Domínguez A, Medina-Bulo I. CMBMeTest: generation of test suites using model-based testing plus constraint programming and metamorphic testing. *Electronics.* 2023;13(1):18.
 38. Amalfitano D, Amatucci N, De Simone V, Riccio V, Rita FA. Towards a thing-in-the-loop approach for the verification and validation of IoT systems. In: Proceedings of the 1st ACM workshop on the internet of safe things; Delft, Netherlands, 2017, New York, NY, USA: ACM, pp. 57–63.
 39. Zafar MN, Afzal W, Enou EP, Haider Z, Singh I. Optimizing model-based generated tests: leveraging machine learning for test reduction. In: International Conference on Software Testing, Verification and Validation Workshops (ICSTW), 2024, Toronto, ON, Canada: IEEE, pp 44-54.
 40. Saadatmand M, Enou EP, Schlingloff H, Felderer M, Afzal W. SmartDelta: automated quality assurance and optimization in incremental industrial software systems development. In: 2022 25th Euromicro conference on digital system design (DSD); 2022.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.